Búsqueda binaria: la mitad y la mitad

Adaptación a C#

Tienes una guía telefónica — de las que la gente solía tener junto al teléfono, ordenada alfabéticamente — y necesitas encontrar un nombre. No la lees de principio a fin. La abres en algún punto por la mitad, ves si el nombre que buscas va antes o después de la página que tienes delante, luego tiras media guía y abres la mitad restante por la mitad. Repite hasta que tengas la página correcta.

Esto es la *búsqueda binaria*. Tiene tres formas: una respuesta directa de "¿está x en la secuencia?", una bisección que devuelve un punto de inserción y — lo más útil — una forma de buscar en el espacio de respuestas de cualquier problema con estructura monótona. Esa tercera forma es la que sigue dando frutos durante el resto del libro.

Cada comparación corta el espacio de búsqueda a la mitad. Para una lista ordenada de un millón de elementos, eso son unas veinte comparaciones para encontrar cualquier cosa; para mil millones de elementos, unas treinta. Duplicar la entrada añade una comparación, no duplica el tiempo. La búsqueda lineal escanearía los mil millones de elementos. La búsqueda binaria lee treinta de ellos y te da la respuesta con certeza.

1. El algoritmo de la mitad y la mitad

La mecánica es exactamente la estrategia de la guía telefónica. Mantienes dos índices, `lo` y `hi`, que marcan los límites de la parte de la secuencia que aún no has descartado. Mientras el intervalo no esté vacío, miras el medio y usas una comparación para tirar la mitad.

```
public static int? BinarySearch<T>(
    T x, IReadOnlyList<T> items, Comparison<T> order = null)
{
    order ??= DefaultOrder;
    int lo = 0, hi = items.Count - 1;
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;
        int cmp = order(x, items[mid]);
        if (cmp == 0)
            return mid;
        else if (cmp < 0)
            hi = mid - 1;
        else
            lo = mid + 1;
    }
    return null;
}
```

Listing 1: Búsqueda binaria iterativa

Las tres ramas hacen exactamente lo que hacía la guía telefónica. Si el elemento del medio es el objetivo, ya está. Si el objetivo es menor, la mitad derecha (todo desde `mid` en adelante) queda descartada, así que `hi` pasa a `mid - 1`. Si el objetivo es mayor, la mitad izquierda queda descartada, así que `lo` pasa a `mid + 1`. El bucle termina cuando `lo >hi` — en ese punto el intervalo está vacío y el objetivo nunca estuvo en la secuencia.

```
var items = new[] { 1, 3, 5, 7, 9, 11, 13, 15 };
Console.WriteLine(Linear.BinarySearch(7,  items)); // 3   (encontrado en
    indice 3)
Console.WriteLine(Linear.BinarySearch(8,  items)); // (null) no presente
Console.WriteLine(Linear.BinarySearch(1,  items)); // 0   primer
    elemento
Console.WriteLine(Linear.BinarySearch(15, items)); // 7   ultimo
    elemento
Console.WriteLine(Linear.BinarySearch(5,  Array.Empty<int>())); // (null
    ) entrada vacia
```

```
3
(null)
0
7
(null)
```

`mid = lo + (hi - lo) / 2` usa división entera, que siempre redondea hacia `lo` — esto es importante para la terminación, porque garantiza que `mid <hi` siempre que `lo <hi`, de modo que la rama `lo = mid + 1` progresa de verdad. (También evita el desbordamiento que tendría `(lo + hi) / 2` cuando `lo + hi` excede `int.MaxValue`.) El otro detalle es que la entrada debe estar ordenada según el orden que define `order`. Si le pasas una lista sin ordenar, la búsqueda binaria te da basura con confianza. No hay forma fácil de que el algoritmo detecte esto — el orden es una precondición, no algo que la función compruebe. Si quieres una red de seguridad, ordena primero y documenta el contrato a gritos.

2. Encontrar puntos de inserción: `BisectLeft` y `BisectRight`

`BinarySearch` responde una pregunta de sí o no con una posición adjunta. Pero hay una pregunta estrechamente relacionada que resulta ser más útil en la práctica: ¿dónde iría este elemento, si lo insertara manteniendo la secuencia ordenada?

Esa posición existe tanto si el elemento ya está presente como si no. Si estás añadiendo una nueva entrada a una lista ya ordenada, no quieres "¿está ahí?" — quieres "¿dónde lo pongo?". Y si el elemento ya está ahí con duplicados, tienes una decisión de contrato que tomar: ¿devuelves la posición antes del primer elemento igual, o después del último?

La convención estándar — la que usa el módulo `bisect` de Python, y la que usaré aquí — te da ambas. `BisectLeft` devuelve el punto más a la izquierda donde podría insertarse el elemento manteniendo la lista ordenada. `BisectRight` devuelve el más a la derecha. Para una lista sin duplicados del objetivo, ambos devuelven la misma respuesta; para una lista con duplicados, delimitan la racha.

```
public static int BisectLeft<T>(
    T x, IReadOnlyList<T> items, Comparison<T> order = null)
{
    order ??= DefaultOrder;
    int lo = 0, hi = items.Count;
    while (lo < hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (order(items[mid], x) < 0)
            lo = mid + 1;
        else
            hi = mid;
```

```
    }
    return lo;
}
```

Listing 2: `BisectLeft`: inserción más a la izquierda

La forma es reconociblemente la de la búsqueda binaria, pero la condición del bucle es `lo <hi` (no `<=`) y `hi` se reduce a `mid` (no `mid - 1`). Eso es porque la respuesta es ahora un índice de inserción, que legalmente puede ser uno más allá del final de la lista — así que el espacio de búsqueda es $[0, \texttt{items.Count}]$, y el bucle converge cuando `lo == hi`.

`BisectRight` es el mismo esqueleto con una comparación invertida.

```
public static int BisectRight<T>(
    T x, IReadOnlyList<T> items, Comparison<T> order = null)
{
    order ??= DefaultOrder;
    int lo = 0, hi = items.Count;
    while (lo < hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (order(items[mid], x) <= 0)
            lo = mid + 1;
        else
            hi = mid;
    }
    return lo;
}
```

Listing 3: `BisectRight`: inserción más a la derecha

El único cambio desde `BisectLeft` es el `<=` en lugar de `<`. Cuando el elemento del medio es igual al objetivo, `BisectLeft` lo trata como "la racha empieza aquí o antes" y se mueve a la izquierda; `BisectRight` lo trata como "la racha termina aquí o después" y se mueve a la derecha. Un bit de diferencia, dos respuestas distintas.

```
var items = new[] { 1, 3, 3, 3, 5, 7 };
Console.WriteLine(Linear.BisectLeft(3,  items)); // 1  (antes de la
    racha de 3s)
Console.WriteLine(Linear.BisectRight(3, items)); // 4  (despues de la
    racha de 3s)
Console.WriteLine(Linear.BisectLeft(4,  items)); // 4  (entre los 3s y
    el 5)
Console.WriteLine(Linear.BisectRight(4, items)); // 4
Console.WriteLine(Linear.BisectLeft(0,  items)); // 0  (menor que todo,
    va al inicio)
Console.WriteLine(Linear.BisectRight(8, items)); // 6  (mayor que todo,
    va al final)
```

```
1
4
4
4
0
6
```

El par (`BisectLeft`, `BisectRight`) no solo responde dónde insertar. También te da, casi gratis, el conteo de un elemento en una secuencia ordenada — simplemente resta: `BisectRight(x, items) - BisectLeft(x, items)`. Eso es $O(\log n)$ en lugar del $O(n)$ `Count` que escribiste en el capítulo anterior. La estructura entrega el doble.

3. Buscar la respuesta, no los datos

Hasta ahora he usado la búsqueda binaria para encontrar algo en una secuencia. Pero el algoritmo real no le importa si tienes una secuencia siquiera. Lo único que necesita es una *comparación monótona* — una pregunta cuya respuesta cambia de un valor a otro exactamente una vez sobre el rango que estás buscando. La parte de "secuencia" era solo una forma de darle esa estructura monótona a un problema.

Aquí está la generalización. Supón que tienes un predicado $P(k)$ — una función que toma un entero y devuelve un booleano — y sabes que P es monótono: hay algún umbral por debajo del cual P es `true` y por encima del cual P es `false`. La búsqueda binaria puede encontrar ese umbral sin que tú materialices nunca una secuencia.

```
/// <summary>
/// Devuelve el mayor entero en [lo, hi] para el cual el predicado es true,
/// asumiendo que el predicado es true en un prefijo [lo, k] y false en [k+1, hi].
/// Devuelve lo - 1 si ningun valor satisface el predicado.
/// </summary>
public static int BinarySearchPredicate(
    Func<int, bool> predicate, int lo, int hi)
{
    while (lo <= hi)
    {
        int mid = lo + (hi - lo) / 2;
        if (predicate(mid))
            lo = mid + 1;
        else
            hi = mid - 1;
    }
    return hi;
}
```

Listing 4: Búsqueda binaria sobre el predicado (*binary search on the answer*)

El cuerpo es casi idéntico a `BinarySearch` — la misma maquinaria de `lo`/`hi`/`mid`, la misma reducción a la mitad — pero en lugar de comparar el objetivo con un elemento de la secuencia, evalúas el predicado en `mid` y usas su respuesta booleana para decidir qué mitad descartar. La función devuelve `hi` al terminar porque, después del bucle, `hi` termina apuntando al mayor satisfacible (la búsqueda se ha estrechado más allá de la frontera).

El uso canónico es la raíz cuadrada entera — el mayor entero k tal que $k^2 \leq n$. El predicado es monótono: al crecer k, $k^2 \leq n$ se mantiene `true` hasta que cruzas la raíz, y luego se mantiene `false` para siempre. La búsqueda binaria encuentra la frontera.

```
public static int IntegerSqrt(int n)
{
    if (n < 0)
        throw new ArgumentOutOfRangeException(nameof(n),
            "integer_sqrt requiere n >= 0");
    return BinarySearchPredicate(k => (long)k * k <= n, 0, n);
}
```

Listing 5: Raíz cuadrada entera vía búsqueda binaria sobre el predicado

```
Console.WriteLine(Linear.IntegerSqrt(10));        // 3   (3*3=9 <= 10 < 16=4*4)
Console.WriteLine(Linear.IntegerSqrt(16));        // 4   (cuadrado exacto)
```

```
Console.WriteLine(Linear.IntegerSqrt(0));         // 0    (frontera)
Console.WriteLine(Linear.IntegerSqrt(1 << 20));   // 1024 (funciona en
    entradas enormes, ~20 iteraciones)
```

```
3
4
0
1024
```

Para $n = 2^{40}$, el rango de búsqueda es $[0, 2^{40}]$, y el bucle termina en unas $\log_2(2^{40}) = 40$ iteraciones. Esa es la aceleración de la búsqueda binaria aplicada a un problema que nunca tuvo una secuencia ordenada en primer lugar — solo una pregunta con estructura monótona.

Este patrón aparece por todas partes. "¿Cuál es el menor número de servidores que aguanta la carga pico?" — monótono. "¿Cuál es la mayor tasa de aprendizaje que converge?" — monótono. "¿Cuál es el máximo ancho de banda que la red puede sostener?" — monótono. Cada vez que la respuesta a una pregunta de sí/no se mantiene igual una vez que cambia, la búsqueda binaria sobre la respuesta está en juego.

4. Las tres preguntas, aplicadas

¿Es correcto?

`BinarySearch` es correcto por un invariante que el bucle preserva: si x está en cualquier parte de `items`, está en algún lugar de `items[lo..hi]`. Eso es cierto al inicio, cuando el intervalo es la secuencia entera. Cada iteración mira `items[mid]`, y las tres ramas preservan el invariante: si `items[mid] == x`, la función retorna; si $x <$ `items[mid]`, entonces x (si está presente) no puede estar en un índice $\geq$ `mid`, así que `hi = mid - 1` sigue siendo un invariante válido; simétrico para la otra rama. Cuando el bucle sale con `lo >hi`, el intervalo está vacío — lo que, por el invariante, significa que x nunca estuvo en `items`.

Invariante de bucle 1. *Si x está en `items`, entonces está en `items[lo..hi]`.*

El mismo argumento de invariante de bucle se traslada a `BisectLeft` (el invariante es "el punto de inserción más a la izquierda está en [`lo`, `hi`]") y a `BinarySearchPredicate` (el invariante es "el umbral está en [`lo` $-$ 1, `hi`]"). Una forma algorítmica, tres contratos estrechamente relacionados.

¿Cuán eficiente es?

Cada iteración corta el intervalo a la mitad. El intervalo empieza con longitud n y se reduce geométricamente: tras k iteraciones, su longitud es como mucho $n/2^k$. El bucle sale cuando la longitud llega a cero, lo que ocurre una vez que $2^k \geq n$ — es decir, $k \geq \log_2 n$. Así que el bucle se ejecuta como máximo $\lceil \log_2 n \rceil$ veces, dando tiempo $O(\log n)$. El espacio es $O(1)$ — solo los dos índices y las locales del bucle.

La reformulación táctil: la búsqueda binaria hace una comparación por cada duplicación de la entrada. Una lista de un millón de elementos tarda unas 20 comparaciones; mil millones tardan unas 30; un quintillón tarda unas 60. El logaritmo es la respuesta a "¿cuántas veces tengo que duplicar 1 antes de llegar a n?" — y para las entradas que te importan en la vida real, esa respuesta siempre es pequeña.

¿Es óptimo?

Este es el golpe del capítulo. Ningún algoritmo de búsqueda basado en comparaciones sobre un arreglo ordenado puede hacerlo mejor que $\lceil \log_2(n+1) \rceil$ comparaciones en el peor caso, y la búsqueda binaria alcanza esa cota exactamente.

El argumento es la cota inferior del *árbol de decisión*. Cualquier algoritmo que solo haga comparaciones puede dibujarse como un árbol de decisión binario: cada nodo interno es una comparación, y cada hoja es uno de los posibles resultados que el algoritmo distingue. Aquí hay $n+1$ resultados posibles — el objetivo está en el índice 0, o en el índice 1, ..., o en el índice $n-1$, o no está presente en absoluto. Un árbol binario de profundidad k tiene como mucho 2^k hojas, así que para encajar $n+1$ hojas necesitas profundidad $\geq \lceil \log_2(n+1) \rceil$. Ese es el camino del peor caso a través del árbol, y por tanto el número de comparaciones del peor caso que cualquier algoritmo basado en comparaciones debe hacer.

Teorema 1. *La búsqueda binaria realiza $\lceil \log_2(n+1) \rceil$ comparaciones en el peor caso y es, por tanto, estrechamente óptima dentro del modelo de comparaciones sobre una secuencia ordenada.*

La búsqueda binaria alcanza $\lceil \log_2(n+1) \rceil$ comparaciones en el peor caso. Es estrechamente óptima — dentro del modelo de comparaciones sobre una secuencia ordenada, no hay mejor algoritmo esperando a ser descubierto.

5. Información por comparación

La búsqueda binaria es correcta porque el invariante "si x está en la secuencia, sigue estando en el intervalo actual" sobrevive a cada iteración. Se ejecuta en tiempo $O(\log n)$ y espacio $O(1)$. Y es óptima: la cota inferior del árbol de decisión dice que ningún algoritmo basado en comparaciones sobre un arreglo ordenado puede hacerlo mejor que $\lceil \log_2(n+1) \rceil$ comparaciones en el peor caso, y la búsqueda binaria cumple esa cota exactamente.

El principio que va más allá de la propia búsqueda binaria: *cada comparación útil debe descartar una fracción constante de las posibilidades restantes.* La búsqueda lineal descarta un elemento por comparación, una fracción que se encoge a cero a medida que n crece. La búsqueda binaria descarta la mitad — una fracción constante, independiente de n. La primera te da $O(n)$; la segunda te da $O(\log n)$. La brecha entre ellas es exactamente la brecha entre información por comparación e información por elemento.

Ese principio va a volver. Cada algoritmo $O(\log n)$ en el resto del libro — los árboles balanceados de la Parte III, las operaciones del montículo, las consultas del árbol de segmentos — se gana su logaritmo descartando una fracción constante del trabajo restante por paso. La búsqueda binaria es el caso más simple; reconocer la misma forma en otros sitios es la mayor parte de la habilidad.

El siguiente capítulo invierte la pregunta: ¿cómo creas el orden que la búsqueda binaria explotó?

6. Código completo

```
using System;
using System.Collections.Generic;

namespace Codex.Search
{
    public static class Binary
    {
        public static int DefaultOrder<T>(T x, T y)
        {
            return Comparer<T>.Default.Compare(x, y);
        }

        // B squeda binaria cl sica. Devuelve el  ndice  o null.
        public static int? BinarySearch<T>(
```

```
            T x, IReadOnlyList<T> items, Comparison<T> order = null)
        {
            order ??= DefaultOrder;
            int lo = 0, hi = items.Count - 1;
            while (lo <= hi)
            {
                int mid = lo + (hi - lo) / 2;
                int cmp = order(x, items[mid]);
                if (cmp == 0) return mid;
                else if (cmp < 0) hi = mid - 1;
                else lo = mid + 1;
            }
            return null;
        }

        // Punto de inserci n m s a la izquierda.
        public static int BisectLeft<T>(
            T x, IReadOnlyList<T> items, Comparison<T> order = null)
        {
            order ??= DefaultOrder;
            int lo = 0, hi = items.Count;
            while (lo < hi)
            {
                int mid = lo + (hi - lo) / 2;
                if (order(items[mid], x) < 0) lo = mid + 1;
                else hi = mid;
            }
            return lo;
        }

        // Punto de inserci n m s a la derecha.
        public static int BisectRight<T>(
            T x, IReadOnlyList<T> items, Comparison<T> order = null)
        {
            order ??= DefaultOrder;
            int lo = 0, hi = items.Count;
            while (lo < hi)
            {
                int mid = lo + (hi - lo) / 2;
                if (order(items[mid], x) <= 0) lo = mid + 1;
                else hi = mid;
            }
            return lo;
        }

        // B squeda binaria sobre el predicado (binary search on the
           answer).
        public static int BinarySearchPredicate(
            Func<int, bool> predicate, int lo, int hi)
        {
            while (lo <= hi)
            {
                int mid = lo + (hi - lo) / 2;
                if (predicate(mid)) lo = mid + 1;
                else hi = mid - 1;
            }
            return hi;
        }
```

```
72
73          // Raíz cuadrada entera vía búsqueda sobre el predicado.
74          public static int IntegerSqrt(int n)
75          {
76              if (n < 0)
77                  throw new ArgumentOutOfRangeException(nameof(n),
78                      "integer_sqrt requiere n >= 0");
79              return BinarySearchPredicate(k => (long)k * k <= n, 0, n);
80          }
81      }
82  }
```

Listing 6: `Binary.cs` — todos los algoritmos

7. Notas y lecturas adicionales

La búsqueda binaria se trata formalmente en CLRS (4.ª ed.) §2.3 en su forma recursiva de divide y vencerás; la forma iterativa usada aquí es la que Knuth analiza en *The Art of Computer Programming* Vol. 3 §6.2.1, donde señala célebremente que la primera búsqueda binaria publicada correctamente apareció en 1962, dieciséis años después de la primera publicada incorrectamente. El módulo `bisect` de la biblioteca estándar de Python es la implementación de referencia canónica para `bisect_left` y `bisect_right` — vale la pena leer el código fuente de CPython por los detalles del off-by-one. La variante basada en predicado bajo el nombre de “binary search on the answer” es el idioma estándar de la programación competitiva; el artículo del sitio CP-Algorithms sobre búsqueda binaria es un buen catálogo de sus variantes.